

Luna

Tech Intern Assessment

Choose one task from the list below and complete it independently. All submissions must follow the instructions in the Submission Requirements section at the end of this brief.

Task 01: Real-Time Wearable Intelligence Dashboard

Build a full-stack system that streams simulated wearable sensor data (heart rate, SpO2, and accelerometer readings) over WebSocket. An ML model evaluates the stream continuously for anomalies. On each detection, a streaming LLM API call is triggered and the generated insight is rendered live on a React-based dashboard.

- Implement real-time animated charts for each metric alongside a live alert feed.
- Display anomaly confidence scores and a timestamp history for every triggered event.
- The backend must handle concurrent WebSocket connections without data loss or stream interruption.
- The LLM response must stream token-by-token into the UI; full-response buffering is not acceptable.

Task 02: Edge AI Model Compression and Deployment

Select an open-source activity or gesture recognition model and apply INT8 post-training quantization. Optionally, apply one additional compression technique such as pruning or knowledge distillation. Deploy the compressed model as a serverless REST endpoint (for example, Hugging Face Inference Endpoints, AWS Lambda, or Google Cloud Functions) so the service incurs no cost when idle and is reachable on demand.

- Benchmark the model across at least three quantization configurations.
- Document accuracy and inference latency trade-offs for each configuration with supporting plots.
- The REST API must accept raw sensor input and return a classified label with a confidence score.
- Setting up automated CI/CD is optional but will be taken into account during review.

Task 03: Autonomous Multi-Tool AI Research Agent

Build an LLM-based agent with at least four functional tools: web search, PDF reader, Python code executor, and a vector memory store. The agent must complete the following research task autonomously: identify the top five competitors in the wearable segment, extract their key technical differentiators, and produce a structured report with supporting charts.

- The agent must detect and recover from individual tool failures without halting execution.
- Cumulative token cost must be tracked and displayed throughout the run.
- All reasoning steps must stream in real time to a frontend UI; no post-hoc display.
- Final output must be a structured, exportable document in PDF or Markdown format.

Task 04: IMU Signal Classification Pipeline

Using a publicly available IMU dataset (UCI HAR, WISDM, or PAMAP2), build a complete machine learning pipeline covering data versioning, feature engineering, multi-model training, and experiment tracking.

- Apply sliding-window segmentation and FFT-based feature extraction as preprocessing steps.
- Train and compare at least three model architectures (e.g. Random Forest, 1D-CNN, Transformer); log all runs to MLflow or Weights & Biases.
- Version the dataset using DVC; all transformations must be fully reproducible from a single command.

- Serve the best-performing model via a REST API.
- Present all results in the most comprehensive and readable manner possible, including per-class metrics, training curves, and a comparative summary across models.

Task 05: Cross-Platform Wearable Health Companion App

Build a mobile application in Kotlin, Flutter, or Swift that connects to a simulated BLE wearable data stream and provides real-time health monitoring with on-device AI inference. The application must run on the platform of your choice (Android or iOS); targeting both platforms is not required.

- Render all incoming health metrics with animated, real-time charts.
- Integrate an on-device ML model (TFLite or Core ML) for step classification or sleep staging; inference must run entirely on-device with no server round-trip.
- Trigger a streaming LLM health insight when the on-device model flags an anomaly.
- The application must support offline mode and persist session history to local storage.

Task 06: Real-Time Computer Vision Workout Form Coach

Build a real-time pose estimation system using MediaPipe or YOLO-Pose that analyses live webcam input to coach workout form, count repetitions, and detect dangerous posture deviations.

- Support up to three exercises (e.g. squat, push-up, deadlift) with per-frame form scoring.
- Render a skeleton overlay on the live feed; trigger a voice alert and a UI notification on any critical posture deviation.
- Serve the experience via a React frontend and log all session metrics (reps, form scores, and timestamps) to a time-series backend store.
- Define and document all joint-angle thresholds and scoring logic in the repository README.

Task 07: Production RAG System with Evaluation Framework

Build a Retrieval-Augmented Generation pipeline over a product knowledge base comprising manuals, FAQs, and specifications. The system must support multiple retrieval strategies and expose a rigorous evaluation framework.

- Implement three retrieval strategies: dense vector search, sparse keyword search, and hybrid retrieval. All three must be configurable at runtime.
- Evaluate all strategies using RAGAS metrics (faithfulness, context recall, and answer relevancy) and surface results in a live performance dashboard.
- Expose a chat interface with inline source citations; every response must display the retrieved document chunks used to generate it.

Submission Requirements

The following are mandatory. Incomplete submissions will not be reviewed.

- Upload all source code to a **public GitHub repository**. The repository must include a `README.md` with clear setup and run instructions. Submit the repository URL via the application form.
- Record a **code walkthrough video** covering the architecture, key design decisions, and how the core components are implemented. Submit the video link via the application form.
- Record a **demo video** of the working product demonstrating all core features described in the task. The demo must show the system running live; no slideshows or static mockups. Submit the video link via the application form.